

Maestría en Ciencia de Datos

Departamento de Matemática y Ciencias

Plan de Trabajo final

Javier Cordeiro

2026

Director: -



Resumen

Agradecimientos

Índice general

Resumen	i
Agradecimientos	ii
Índice general	iii
1 Introducción	1
1.1. Objetivo	1
1.2. Situación actual	1
1.3. Algoritmo actual	1
1.4. Implementación del trabajo final	1
2 Marco teórico	4
Bibliografía	5

CAPÍTULO 1

Introducción

1.1. Objetivo

El objetivo de este trabajo es desarrollar un modelo de programación lineal para verificar la efectividad del algoritmo de "batching" utilizando actualmente en la empresa Shiphero. El alcance del trabajo no va ser reemplazar al algoritmo, sino verificar las diferencias y encontrar puntos de mejora.

1.2. Situación actual

Shiphero desarrolla un software para la gestión de depósitos, que incluye funciones como la gestión de stock, el armado de pedidos y el despacho de mercadería.

El caso en el que se va a enfocar en el trabajo es el de batching y routing. Batching se denomina al proceso en el cual un operario del deposito debe buscar los productos para armar un grupo de pedidos, en este caso 25. Los productos están almacenados en estanterías, de manera relativamente aleatoria y el objetivo es de los pedidos pendientes encontrar los 25 que estén más cercanos entre si para lograr reducir el tiempo de trabajo. Ésta tarea es clave ya que se estima que el las tareas de recolección de órdenes pueden ser del order del 50 % al 70 % de los costos de operación del depósito.

La modalidad de trabajo de Shiphero es del tipo 3PL (third party logistics), la cual consiste en en la la empresa se encarga de la gestión del stock, el armado de pedidos y el despacho de mercadería de multiples clientes. Las ordenes no mezclan productos de distintos clientes pero en el batching se pueden mezclar ordenes de distintos clientes.

1.3. Algoritmo actual

El algoritmo que resuelve el batching se ejecuta cada vez que un operario solicita una acción de batching y determina las 25 ordenes más cercanas entre si entre todos las órdenes pendientes en ese momento.

1.4. Implementación del trabajo final

La implementación del trabajo final se va a realizar en Python, utilizando la librería Pulp para la resolución de los modelos de programación lineal. La

1. Introducción

Algorithm 1 Algoritmo de Batching

```
1: Cargar todas las ordenes pendientes con sus respectivos items y coordenadas
   en el depósito
2: for EACH order_id en orders_to_process do
3:   Filtrar las ordenes = order_id
4:   Ordenar por bay
5:   Calcular la cantidad de bays
6:   if  $bays \leq 5$  then
7:      $path \leftarrow$  coordenadas de todos las bays
8:   else
9:     Simplifico el recorrido calculando 5 puntos claves: Inicio, primer cuarto,
       mitad tercer cuarto y final.
10:     $qr_{ix} \leftarrow$  Indice del cuartil
11:     $mid_{ix} \leftarrow$  Indice de la mitad
12:     $path \leftarrow [0, qr_{ix}, mid_{ix}, 2 \times qr_{ix}, n_{bays} - 1]$ 
13:  end if
14:  orders_path.APPEND([order_id, path])
15: end for
16: Cálculo de la distancia de Hausdorff
17:  $n\_ordenes \leftarrow size(orders\_path)$ 
18:  $matriz\_dist\_ordenes \leftarrow n\_ordenes \times matriz\_n\_ordenes$ 
19: for  $i = 0$  to  $n\_ordenes - 1$  do
20:    $path\_A \leftarrow orders\_path[i][1]$ 
21:   for  $j = 0$  to  $n\_ordenes - 1$  do
22:      $path\_B \leftarrow orders\_path[j][1]$ 
23:      $dist\_AB \leftarrow Hausdorff(path\_A, path\_B)$ 
24:      $dist\_BA \leftarrow Hausdorff(path\_B, path\_A)$ 
25:     Distancia simétrica de Hausdorff
26:      $dist\_final \leftarrow max(dist\_AB, dist\_BA)$ 
27:      $matriz\_dist\_ordenes[i][j] \leftarrow dist\_final$ 
28:      $matriz\_dist\_ordenes[j][i] \leftarrow dist\_final$ 
29:   end for
30: end for
31: Generación de batches greedy
32:  $batches \leftarrow$  lista vacía
33:  $n\_batches \leftarrow floor(n\_ordenes / ORDENES\_POR\_BATCH)$ 
34: for  $k \leftarrow 1$  to  $n\_batches$  do
35:   Encontrar la orden semilla con los mejores vecinos más cercanos
36:    $distancias\_ordenadas \leftarrow$  Ordeno cada fila de orders_distance_matrix
37:   Obtengo la matriz de distancias total para cada una de las órdenes más
   cercanas
38:    $dist\_ordenes\_cercanas \leftarrow$  Primera columna de  $ordenes\_per\_batch\_i$  de  $distancias\_ordenadas$ 
39:    $suma\_distancia\_ordenes \leftarrow sum(dist\_ordenes\_cercanas, axis = 1)$ 
40:   Elijo la orden con la menor suma
41:   Armo un batch con esa orden y sus 24 más cercanas
42:   Elimino la orden de la matriz y repito
43: end for
```

idea es generar un modelo que permita comparar los resultados obtenidos por el algoritmo actual con los resultados obtenidos por el modelo de programación lineal. Como el objetivo es comparar los resultados, se va a utilizar la misma información que utiliza el algoritmo actual para generar los batches. La información que se va a utilizar es la siguiente:

- Lista de ordenes pendientes con sus respectivos items y coordenadas en el depósito.
- Cantidad de ordenes por batch (25).
- Cantidad de batches a generar (1).
- Distancia de Hausdorff entre cada par de ordenes.
- Matriz de distancias entre cada par de ordenes.
- Lista de ordenes generadas por el algoritmo actual.
- Lista de ordenes generadas por el modelo de programación lineal.
- Tiempo de ejecución del algoritmo actual.
- Tiempo de ejecución del modelo de programación lineal.
- Cantidad de ordenes generadas por el algoritmo actual.
- Cantidad de ordenes generadas por el modelo de programación lineal.
- Distancia total recorrida por el algoritmo actual.
- Distancia total recorrida por el modelo de programación lineal.

Como segundo objetivo, si el tiempo de preparación de la tesis lo permite, se verificará cómo afecta los tiempos y distancias la reorganización de los productos en el depósito. Actualmente los productos están almacenados en estanterías de manera relativamente aleatoria, se podría verificar de organizarlos por dándole una sector a cada vendedor y dejar los que mas venden en la parte más cercana a las estaciones de empaquetado.

CAPÍTULO 2

Marco teórico

El problema abordado en este trabajo se enmarca dentro de la gestión operativa de depósitos, en particular en las decisiones asociadas al *order picking*, es decir, el proceso mediante el cual los operarios recorren el depósito para recolectar los productos necesarios para preparar órdenes de clientes. Esta actividad suele representar una proporción significativa del costo operativo del depósito, por lo que pequeñas mejoras en la selección de órdenes, el agrupamiento y el recorrido pueden tener impacto directo sobre la productividad.

Dentro del *order picking*, el *batching* consiste en agrupar varias órdenes para que sean recolectadas en un mismo recorrido. El objetivo es formar conjuntos de órdenes que compartan ubicaciones cercanas o recorridos compatibles, reduciendo así la distancia total recorrida y el tiempo de preparación. En contextos 3PL, donde un operador logístico administra inventario y pedidos de múltiples clientes, esta decisión es especialmente relevante porque el sistema debe coordinar órdenes heterogéneas sin comprometer la eficiencia operativa.

El problema de *batching* puede abordarse mediante heurísticas o mediante modelos de optimización. Las heurísticas, como los algoritmos greedy, permiten obtener soluciones rápidas y son adecuadas para entornos operativos donde el tiempo de respuesta es crítico. Sin embargo, no garantizan optimalidad y pueden dejar oportunidades de mejora sin explorar. En contraste, los modelos de programación lineal entera permiten formular explícitamente una función objetivo y un conjunto de restricciones, obteniendo una solución óptima o una cota contra la cual comparar el desempeño del algoritmo actual.

En este trabajo, la distancia entre órdenes se representa a partir de sus ubicaciones dentro del depósito. Para ello se utiliza una matriz de distancias entre pares de órdenes, construida mediante la distancia de Hausdorff entre los recorridos aproximados de cada orden. Esta métrica permite comparar conjuntos de puntos y resulta útil cuando cada orden no está representada por una única ubicación, sino por varias posiciones asociadas a los productos que la componen.

La propuesta se vincula con la literatura reciente sobre modelos de programación lineal aplicados a problemas de *batching* y balanceo de carga en depósitos. En particular, Tarczyński, 2023 muestra que los modelos de programación lineal entera pueden utilizarse para optimizar decisiones de preparación de pedidos y comparar políticas operativas alternativas. Siguiendo esta línea, el presente trabajo no busca reemplazar directamente el algoritmo productivo de Shiphero, sino construir un modelo de referencia que permita evaluar su desempeño, cuantificar diferencias y detectar posibles puntos de mejora.

Bibliografía

Tarczyński, G. (2023). Linear programming models for optimal workload and batching in pick-and-pass warehousing systems. *Operations Research and Decisions*, 33(3). <https://doi.org/10.37190/ord230309>